

Ripasso Rapido C

Argomenti: Bitwise, Precedenze, Storage Classes, Puntatori, Preprocessore, Modularità, CLI

1. Operatori Bitwise

Scopo: Manipolare singoli bit di interi. Usare unsigned per gli shift logici per evitare comportamenti anomali col segno.

- & (AND) — | (OR) — ^ (XOR) — ~ (NOT / Complemento a 1)
- << (Shift sinistra) — Moltiplica per potenze di 2
- >> (Shift destra) — Divide per potenze di 2

Snippet da memorizzare:

C

```
if (n & 1)           // Verifica se n è dispari
x |= (1 << k);       // Imposta a 1 il bit k
x &= ~(1 << k);      // Azzera (imposta a 0) il bit k
x ^= (1 << k);       // Inverte (toggle) il bit k
y = x & 0xFF;        // Prende solo gli 8 bit meno
                    // significativi (LSB)
```

2. Precedenze e Associatività

Regola d'oro: Quando in dubbio, usa le parentesi.

- **Postfix** (a++, (), []): Precedenza massima.
- **Unari** (-, !, ~, *, &): Valutati subito dopo; associatività da *destra verso sinistra*.
- **Ordine generale:** Postfix → Unari → Moltiplicativi → Additivi → Shift → Bitwise → Confronti → Logici → Assegnamenti.

Esempio insidioso:

C

```
int a = 5;
printf("%d\n", -a++); // Stampa -5, poi la variabile 'a' diventa 6
```

3. Classi di Memorizzazione (Storage Classes)

Keyword	Lifetime	Scope / Uso
auto	Blocco	Locale; default per le variabili locali.
static	Esecuzione	Locale al file; mantiene il valore tra le chiamate.

Keyword	Lifetime	Scope / Uso
extern	Esecuzione	Globale; variabile definita in un altro modulo.

Note Rapide: la keyword **static** locale viene inizializzata una sola volta. Se non inizializzata esplicitamente, parte da 0. **extern** non alloca memoria; la definizione reale deve esistere in un file .c.

4. Puntatori Essenziali

- &x (Indirizzo), *p (Dereferenziazione), NULL (Puntatore nullo).
- **Decadimento Array:** Un array decade a puntatore al suo primo elemento (int *p = v; equivale a int *p = &v[0];).
- **Aritmetica:** Spostare un puntatore (p + 1) lo avanza di sizeof(*p) byte.
- Puntatore a puntatore (p): Necessario per modificare un puntatore passandolo a una funzione.

Pattern: Swap per array di stringhe



```
void swap(char **a, char **b) {
    char *t = *a;
    *a = *b;
    *b = t;
}
```

Pattern: Puntatore a funzione



```
int (*fp)(int, float); // Dichiarazione
fp = nome_funzione;    // Assegnazione
int r = fp(3, 2.0);     // Chiamata
```

5. Preprocessore e Macro

Le direttive principali avvengono prima della compilazione vera e propria: #include, #define, #undef, #if, #ifdef, #ifndef, #else, #endif.

Macro sicure (Usa sempre le parentesi con #define):



```
#define SQ(x) ((x) * (x))
#define MIN(a,b) ((a) < (b)) ? (a) : (b)
```

- **Macro predefinite:** FILE, LINE, DATE, TIME, STDC.
- *Consiglio:* Evita logiche troppo complesse nelle macro; preferisci funzioni static inline.

6. Modularità e Compilazione Separata

- **Header (.h):** Contengono prototipi, typedef, macro e dichiarazioni extern.
- **Sorgenti (.c):** Contengono le implementazioni vere e proprie.

Protezione degli Header (Include Guards):

C

```
#ifndef MIO_HEADER_H
#define MIO_HEADER_H
// Contenuto dell'header
#endif
```

Comandi CLI Base:

Bash

```
gcc -c file.c      # Compila e genera il file oggetto file.o
gcc -o prog *.o    # Linka i file oggetto in un eseguibile 'prog'
```

- **Makefile:** Fondamentale per automatizzare dipendenze e ricompilazione parziale. Includere sempre un target clean.

7. Argomenti Main e Environment

La firma standard del main per leggere argomenti da riga di comando:

C

```
int main(int argc, char *argv[]) {
    // argv[0] contiene sempre il nome del programma lanciato
    // da argv[1] a argv[argc-1] ci sono gli argomenti veri e
    propri
    return 0;
}
```

- **Environment:** Variabili NOME=VALORE ereditate dal sistema operativo (es. processo padre). Si leggono in C dichiarando extern char environ;.